

Kalvita: Layers & How It Works

Extended technical guide for the Kalvita prototype and its eventual bytecode / virtual-machine architecture.

Kalvita can be viewed as a set of layers. Each layer transforms information into a form the next layer understands. This separation makes the language easier to build, debug, and extend.

```
.kal source
↓
Header validation
↓
Lexer / lexical analysis
↓
Tokens
↓
Parser / syntax analysis
↓
AST (Abstract Syntax Tree)
↓
Semantic + type checking
↓
Interpreter Bytecode compiler
↓ ↓
execution .kalc
↓
Kalvita VM
↓
CPU
```

Prototype: .kal → header → lexer → parser → AST → interpreter. The .kalc compiler and VM can be added later.

1. Source and Header

A human writes Kalvita in a .kal file. The header identifies the script and its language version.

```
[SCRIPTTYPE KALVITA VERSION 1]

kal.OnStart {
var local wow string = "Hello world"
con.Print(wow)
}
```

Header validation can reject a missing, malformed, or unsupported version before normal parsing begins.

2. The Lexer

The lexer (lexical analyzer) reads characters and converts them into tokens. It answers: **what pieces are present?** It does not execute the program or decide the full meaning.

```
var local wow string = "Hello world"
↓
VAR LOCAL IDENTIFIER(wow) TYPE(string) EQUAL STRING("Hello world")
```

Tokens should retain source location information such as line and column. That allows later diagnostics to identify the culprit location.

3. Lexer Lookahead

Kalvita's `./.` comment syntax requires lookahead because a normal `'.'` is also used for member access such as `con.Print()`. When the lexer sees `'.'`, it checks the following characters.

```
.
↓ look ahead
./
↓ look ahead again
.//
↓
comment start
```

If the sequence is `./.`, the lexer enters comment-skipping mode. Otherwise it emits a normal DOT token. The same idea can be used for `==`, `===`, `!=`, `!==`, `!===`, `<=`, and `>=`.

4. Comments

Comments are recognized and discarded by the lexer. The parser normally receives no comment tokens and therefore knows nothing about comments.

```
././ ignored by the parser //

kal.OnStart {
  ././ also ignored //
  con.Print("Hello")
}
```

The lexer still has to find the closing `//` so it can detect an unterminated comment.

```
././ comment with no ending
→ Lexer Error: Unterminated comment.
```

5. Lexer vs Parser Errors

Error	Layer	Meaning	Example
Lexer	Lexer	Characters cannot become valid tokens.	Unexpected @
Syntax	Parser	Tokens are valid, but their structure is invalid.	Missing)
Type	Semantic checker	Types are incompatible.	number = string
Semantic	Semantic checker	A valid structure violates a language rule.	Unknown variable
Runtime	Interpreter / VM	Failure occurs during execution.	Invalid runtime operation

6. The Parser

The parser receives tokens and checks whether they follow Kalvita's grammar. If they do, it constructs an Abstract Syntax Tree (AST).

```
kal.OnStart {
  con.Print("Hello")
}
```

The parser can understand this as an event belonging to the `kal` object, containing a call to `con.Print`. It should not execute that call.

7. Grammar

A grammar describes valid arrangements of tokens. A simplified model is:

```
event := identifier "." identifier block
block := "{" statement* "}"
statement := variable | function_call | event | ...
```

```
function_call:= identifier "." identifier "(" arguments? ")"
```

This is only a conceptual grammar. The real Kalvita grammar can grow with conditions, loops, arrays, functions, objects, processes, and more.

8. AST

The AST is the structured representation produced by the parser. It removes irrelevant source details while keeping the information required by the interpreter or compiler.

```
Event
■■■ object: kal
■■■ name: OnStart
■■■ body
■■■ FunctionCall
■■■ object: con
■■■ function: Print
■■■ argument: "Hello"
```

9. Semantic and Type Checking

Once syntax is valid, Kalvita can check whether the program makes sense. This includes names, scopes, function calls, operations, and types.

```
var local x number = "Hello"
```

```
Syntax: valid
Meaning/type: invalid
→ Type error: Expected number, got string.
```

Semantic checking is different from parsing: parsing checks structure; semantic/type checking checks whether that valid structure obeys Kalvita's rules.

10. The Interpreter Prototype

For the first prototype, execute the AST directly. This avoids designing .kalc and the virtual machine at the same time.

```
.kal
↓
Lexer
↓
Parser
↓
AST
↓
Interpreter
↓
Kalvita runtime / OS APIs
```

A `con.Print()` AST node can call an internal console implementation. Later, the same operation can instead be compiled into a bytecode instruction.

11. Values and Variables

The runtime needs representations for Kalvita values. An early prototype can start with null, number, string, and logic.

```
KalValue
■■■ Null
■■■ Number
■■■ String
■■■ Logic
```

A variable environment maps names to values and tracks scopes. Local and global variables can be implemented as different environments or as environments with scope metadata.

12. Functions and `kal.return()`

Functions can contain parameters and a body. `kal.return()` transfers control back to the caller and can optionally carry a value.

```
function add(a, b) {
  kal.return(a + b)
}

result = add(5, 3)
```

A call stack is useful because nested function calls need to remember where execution should resume.

13. Events

Events are blocks associated with objects or system components. The parser represents their structure; the runtime decides when to trigger them.

```
kal.OnStart {
  con.Print("Started")
}

con.OnExit {
  ...
}

con.OnPrint {
  ...
}
```

This also supports object-specific events such as `window1.OnStart` or `process1.OnStart`.

14. Console Runtime

The `con` object can expose console operations such as `Print()`, `Clear()`, `SetTextColor()`, and `SetConsoleColor()`.

```
con.Print("Hello")
con.Clear()
con.SetTextColor([255, 255, 255])
con.SetConsoleColor([200, 200, 200])
```

Your design distinguishes normal `print()` from `con.Print()`: `con.Print()` triggers `con.OnPrint`, while normal `print()` does not. That behavior belongs to the runtime/event system.

15. Windows and Processes

Windows and processes can be different runtime object types while sharing some methods. The parser can use generic `object.member` syntax instead of needing a separate parser for every object.

```
window1.OnStart {
window1.SetPosition([100, 100])
}

process1.OnStart {
con.Print("Process started")
}
```

At runtime, the object determines which implementation receives the operation. This allows shared methods without pretending that windows and processes are the same type.

16. Error System: error.rs

A shared error module can define the diagnostic structure used by the lexer, parser, semantic checker, and runtime.

```
Diagnostic
■■■ level: Error / Warning
■■■ kind: Lexer / Syntax / Type / Runtime / ...
■■■ message
■■■ file
■■■ line
■■■ column
■■■ section
```

This supports the detailed culprit-line style you want for Kalvita diagnostics. Fatal errors can stop the current operation; warnings can normally allow execution to continue.

17. Rust Result Flow

Rust's `Result` type fits this pipeline well.

```
let tokens = lexer.tokenize()?;
let ast = parser.parse(tokens)?;
let result = interpreter.run(ast)?;
```

The `?` operator immediately propagates an error. Thus a fatal lexer or parser error prevents invalid data from reaching the next stage.

18. Bytecode Compiler

After the interpreter works, add a compiler that walks the AST and emits Kalvita bytecode.

```
AST
↓
Bytecode compiler
↓
.kalc
↓
Kalvita VM
```

The compiler receives structured AST nodes rather than raw source text. This is why the lexer/parser work remains useful even after the interpreter is replaced or supplemented by a VM.

19. .kalc Structure

A future `.kalc` format could contain a magic identifier, bytecode format version, runtime/language version, flags, constants, instructions, function metadata, event metadata, and optional debugging/source metadata.

.kalc

- magic / identifier
- bytecode version
- Kalvita version
- flags
- constants
- instructions
- function metadata
- event metadata
- optional debug metadata

Versioning lets a future VM reject incompatible bytecode cleanly.

20. SHA-256 and Bytecode Caching

A compiled .kalc cache can store a SHA-256 hash of the source. When the source is run again, Kalvita can compare the current hash with the stored one.

```
main.kal
↓ SHA-256
stored source hash
↓ compare
unchanged + compatible → reuse .kalc
changed / incompatible → compile again
```

This gives .kalc a role similar to cached bytecode while still allowing the source file to remain the main human-readable form.

21. Kalvita Virtual Machine

The VM executes bytecode instructions. A stack-based VM is a practical design for an early implementation.

```
Instruction pointer
↓
current instruction
↓
stack
↓
locals / runtime objects
```

A conceptual addition might compile into:

```
LOAD_CONST 10
LOAD_CONST 20
ADD
STORE_LOCAL result
```

22. Interpreter vs VM

	Interpreter	Bytecode VM
Input	AST	.kalc
First prototype	Excellent	Later
Execution	Walk AST nodes	Execute instructions
Caching	Not the main goal	Natural with .kalc

23. Recommended Rust Project Structure

```
kalvita/
├── Cargo.toml
├── src/
│   ├── main.rs
│   ├── error.rs
│   ├── token.rs
│   ├── lexer.rs
│   ├── ast.rs
│   ├── parser.rs
│   ├── value.rs
│   ├── environment.rs
│   ├── interpreter.rs
│   └── runtime/
│       └── mod.rs
```

- ■■■ console.rs
- ■■■ window.rs
- ■■■ process.rs
- compiler/ (later)
- mod.rs
- bytecode.rs
- vm.rs

The exact organization is flexible. The important part is keeping language parsing, runtime behavior, and later compilation reasonably separated.

24. Recommended Prototype Build Order

#	Component	Goal
1	error.rs	Basic diagnostics: message, kind, line, column, section.
2	token.rs	Token kinds and source locations.
3	lexer.rs	Whitespace, comments, identifiers, strings, numbers, punctuation and operators.
4	ast.rs	Expressions, statements, blocks, calls and events.
5	parser.rs	Turn tokens into an AST.
6	value.rs	Null, number, string and logic.
7	environment.rs	Variables and scopes.
8	interpreter.rs	Execute the AST.
9	console runtime	con.Print(), Clear(), colors and console events.
10	functions/events	Functions, kal.return(), OnStart and related events.
11	semantic/type checks	Names, scopes, types and valid operations.
12	windows/processes	Runtime objects and shared operations.
13	compiler	AST → bytecode.
14	.kalc + VM	Serialize and execute bytecode.

25. Useful Debugging Commands

```
kalvita --tokens main.kal
kalvita --ast main.kal
kalvita --trace main.kal
kalvita main.kal
```

--tokens helps debug the lexer, --ast helps debug the parser, --trace helps debug interpreter execution, and the normal command runs the program.

26. The First Real Milestone

A strong first milestone is getting this program to run:

```
[SCRIPTTYPE KALVITA VERSION 1]

kal.OnStart {
var local wow string = "Hello world"
con.Print(wow)
}
```

Expected output:

```
Hello world
```

If that works, the prototype already has source loading, header handling, lexing, parsing, AST construction, variables, an event, and a console operation.

27. Complete Mental Model

```

Human writes .kal
↓
Header validation
↓
Lexer: "What are these characters?"
↓
Tokens
↓
Parser: "Is this valid syntax?"
↓
AST
↓
Semantic/type checker: "Does this make sense?"
↓
████████████████████████████████████████
↓ ↓
Interpreter Bytecode compiler
↓ ↓
execute .kalc
↓
Kalvita VM
↓
runtime
↓
OS
↓
CPU

```

28. Final Design Principle

Keep each layer doing one job. The lexer should not parse program structure. The parser should not execute code. The semantic checker should not tokenize source. The bytecode compiler should not rediscover syntax. The VM should not need to understand human-readable .kal.

This lets a small Rust prototype grow into the larger Kalvita system without rewriting every component whenever a new feature is added.